

LAPORAN PRAKTIKUM PEKAN 5

STRUKTUR DATA

Disusun Oleh:

Silva Nurifvan

2511533004

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS PADANG

2026

KATA PENGANTAR

Assalamu'alaikum warahmatullahi wabarakatuh,

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW. yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini. Penulis mengucapkan terima kasih kepada bapak Dr. Wahyudi, S.T, M.T. selaku dosen mata kuliah Struktur Data yang telah memberikan tugas ini sehingga dapat menambah pengetahuan dan wawasan sesuai dengan mata kuliah yang penulis tekuni. Penulis juga mengucapkan terima kasih kepada semua pihak yang telah membagi sebagian pengetahuannya sehingga penulis dapat menyelesaikan Laporan Praktikum ini.

Padang, 6 Mei 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, struktur data digunakan untuk mengatur dan mengelola data agar dapat diakses serta dimanipulasi dengan efisien. Salah satu struktur data yang penting adalah Single Linked List (SLL), yaitu kumpulan node yang saling terhubung melalui pointer. Setiap node terdiri dari dua bagian utama, yaitu data dan referensi (next) yang menunjuk ke node berikutnya. Pembentukan dasar dari struktur ini direpresentasikan melalui class NodeSLL, yang berfungsi sebagai elemen penyusun linked list. Node ini menjadi fondasi utama dalam membangun hubungan antar data secara berantai.

Untuk melakukan manipulasi data, digunakan class TambahSLL yang berfungsi untuk menambahkan node ke dalam linked list, baik di bagian depan, belakang, maupun pada posisi tertentu. Proses ini menunjukkan bagaimana data dapat dimasukkan secara dinamis tanpa perlu menggeser elemen lain seperti pada array. Selain itu, terdapat class PencarianSLL yang digunakan untuk melakukan pencarian data dalam linked list. Proses pencarian dilakukan dengan menelusuri setiap node secara berurutan hingga data yang dicari ditemukan atau hingga mencapai akhir list. Selanjutnya, class HapusSLL digunakan untuk menghapus node dari linked list, baik pada bagian awal, akhir, maupun pada posisi tertentu. Proses ini melibatkan perubahan pointer antar node sehingga struktur list tetap terhubung dengan benar setelah penghapusan dilakukan.

Dengan memahami konsep NodeSLL serta operasi penambahan, pencarian, dan penghapusan pada Single Linked List, dapat diperoleh pemahaman yang lebih mendalam mengenai pengelolaan data secara dinamis dalam pemrograman.

1.2 Tujuan

1. Memahami konsep dasar struktur data Single Linked List.
2. Mampu mengimplementasikan operasi penambahan node pada SLL (di depan, belakang, dan posisi tertentu).
3. Mampu mengimplementasikan operasi penghapusan node pada SLL (di awal, akhir, dan posisi tertentu).
4. Mampu melakukan pencarian data dalam Single Linked List.
5. Memahami cara kerja pointer dalam menghubungkan antar node pada linked list.

1.3 Manfaat Praktikum

1. Mahasiswa dapat memahami cara kerja struktur data dinamis secara lebih mendalam.
2. Mahasiswa mampu mengelola data secara fleksibel tanpa bergantung pada ukuran tetap seperti pada array.
3. Melatih kemampuan logika dalam mengatur hubungan antar node menggunakan pointer.
4. Menjadi dasar untuk mempelajari struktur data yang lebih kompleks seperti doubly linked list, stack, queue, dan tree.

5. Membantu dalam pengembangan program yang membutuhkan manipulasi data secara efisien

BAB II

PEMBAHASAN

DASAR TEORI

Struktur data merupakan cara untuk menyimpan, mengatur, dan mengelola data di dalam komputer agar data dapat diproses secara efisien. Penggunaan struktur data yang tepat dapat mempermudah proses pengolahan data seperti penambahan, penghapusan, pencarian, dan penelusuran data. Salah satu struktur data linear yang sering digunakan adalah Single Linked List. Single Linked List adalah struktur data yang terdiri dari kumpulan node yang saling terhubung satu arah. Setiap node memiliki dua bagian utama, yaitu data dan pointer next yang berfungsi untuk menunjuk node berikutnya. Node pertama disebut head, sedangkan node terakhir akan menunjuk ke null yang menandakan akhir dari linked list.

Single Linked List memiliki sifat dinamis karena ukuran data dapat bertambah atau berkurang selama program berjalan. Hal ini berbeda dengan array yang ukuran datanya bersifat tetap. Dengan menggunakan linked list, proses penambahan dan penghapusan data dapat dilakukan dengan lebih fleksibel karena tidak memerlukan pergeseran elemen seperti pada array. Beberapa operasi dasar pada Single Linked List antara lain insert, delete, traversal, dan searching. Operasi insert digunakan untuk menambahkan node baru ke dalam linked list, baik di awal, akhir, maupun posisi tertentu. Operasi delete digunakan untuk menghapus node dari linked list. Operasi traversal dilakukan dengan menelusuri seluruh node dari head hingga node terakhir untuk menampilkan data. Sedangkan searching digunakan untuk mencari data tertentu dengan menelusuri node satu per satu sampai data ditemukan atau mencapai akhir linked list.

Dalam implementasinya, Single Linked List menggunakan pointer untuk menghubungkan setiap node. Pointer next menyimpan alamat node berikutnya sehingga node-node pada linked list dapat saling terhubung dan membentuk suatu rangkaian data yang berurutan. Implementasi struktur data dapat dibuat menggunakan Java. Java merupakan bahasa pemrograman berorientasi objek yang mendukung penggunaan class dan object sehingga mempermudah pembuatan node serta pengelolaan operasi pada Single Linked List.

2.1 NodeSLL_2511533004

Kode Program

```
1 package pekan5_2511533004;
2
3 public class NodeSLL_2511533004 {
4     // node bagian data
5     int data_3004;
6     // Pointer ke node berikutnya
7     NodeSLL_2511533004 next_3004;
8     // Konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511533004(int data_3004)
10    {
11        this.data_3004 = data_3004;
12        this.next_3004 = null;
13    }
14
15 }
16
17
```

Langkah Kerja

Data_3004 untuk menyimpan isi data seperti angka dan next_3004 sebagai petunjuk ke node berikutnya.

```
// node bagian data
int data_3004;
// Pointer ke node berikutnya
NodeSLL_2511533004 next_3004;
```

Fungsi yang otomatis jalan saat bikin node baru.

```
// Konstruktor menginisialisasi node dengan data
public NodeSLL_2511533004(int data_3004)
{
    this.data_3004 = data_3004;
```

Data yang dimasukkan disimpan ke node.

```
{
    this.data_3004 = data_3004;
    this.next_3004 = null;
}
```

2.2 TambahSLL_2511533004

Kode Program

```
1 package pekan5_2511533004;
2
3 public class TambahSLL_2511533004 {
4     public static NodeSLL_2511533004 insertAtFront(NodeSLL_2511533004 head, int value) {
5         NodeSLL_2511533004 new_node = new NodeSLL_2511533004(value);
6         new_node.next_3004 = head;
7         return new_node;
8     }
9     // fungsi menambahkan node di akhir SLL
10    public static NodeSLL_2511533004 insertAtEnd(NodeSLL_2511533004 head, int value) {
11        // buat sebuah node dengan sebuah nilai
12        NodeSLL_2511533004 newNode = new NodeSLL_2511533004(value);
13        // jika list kosong maka node jadi head
14        if (head == null) {
15            return newNode;
16        }
17        // simpan head ke variabel sementara
18        NodeSLL_2511533004 last = head;
19        // telusuri ke node akhir
20        while (last.next_3004 != null) {
21            last = last.next_3004;
22        }
23        // ubah pointer
24        last.next_3004 = newNode;
25        return head;
26    }
27    static NodeSLL_2511533004 GetNode(int data_3004) {
28        return new NodeSLL_2511533004(data_3004);
29    }
30    static NodeSLL_2511533004 insertPos(NodeSLL_2511533004 headNode, int position, int value) {
31        NodeSLL_2511533004 head = headNode;
32
33        if (position < 1) {
34            System.out.print("Invalid position");
35        }
36
37        if (position == 1) {
38            NodeSLL_2511533004 new_node = new NodeSLL_2511533004(value);
39            new_node.next_3004 = head;
40            return new_node;
41        } else {
42            while (position-- != 0) {
43                if (position == 1) {
44                    NodeSLL_2511533004 newNode = GetNode(value);
45                    newNode.next_3004 = headNode.next_3004;
46                    headNode.next_3004 = newNode;
47                    break;
48                }
49                headNode = headNode.next_3004;
50            }
51
52            if (position != 1) {
53                System.out.print("Posisi di luar jangkauan");
54            }
55            return head;
56        }
57    }
58    public static void printList(NodeSLL_2511533004 head) {
59        NodeSLL_2511533004 curr = head;
60        while (curr.next_3004 != null) {
61            System.out.print(curr.data_3004 + "-->");
62            curr = curr.next_3004;
63        }
64
65        if (curr.next_3004 == null) {
66            System.out.print(curr.data_3004);
67        }
68        System.out.println();
69    }
70    public static void main(String[] args) {
71        // buat linked list 2->3->5->6
72        NodeSLL_2511533004 head = new NodeSLL_2511533004(2);
73        head.next_3004 = new NodeSLL_2511533004(3);
74        head.next_3004.next_3004 = new NodeSLL_2511533004(5);
75        head.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(6);
76        // cetak list asli
```

```

77     System.out.print("Senarai berantai awal:");
78     printList(head);
79     // tambahkan node baru di depan
80     System.out.print("tambah 1 simpul di depan: ");
81     int data = 1;
82     head = insertAtFront(head, data);
83     // cetak update list
84     printList(head);
85     // tambahkan node baru di belakang
86     System.out.print("tambah 1 simpul di belakang: ");
87     int data2 = 7;
88     head = insertAtEnd(head, data2);
89     // cetak update list
90     printList(head);
91     System.out.print("tambah 1 simpul ke data 4: ");
92     int data3 = 4;
93     int pos = 4;
94     head = insertPos(head, pos, data3);
95     // cetak update list
96     printList(head);
97 }
98 }

```

Langkah Kerja

Fungsi ini untuk menambahkan node di depan list dengan membuat node baru dengan isi value_3004, lalu node baru diarahkan ke head lama dan node baru menjadi head baru

```

public static NodeSLL_2511533004 insertAtFront(NodeSLL_2511533004 head,
NodeSLL_2511533004 new_node = new NodeSLL_2511533004(value);
new_node.next_3004 = head;
return new_node;
}
// fungsi menambahkan node di akhir SLL

```

Fungsi untuk menambahkan node di akhir list dengan membuat node baru.

```

// tambahkan node ke akhir SLL
public static NodeSLL_2511533004 insertAtEnd(NodeSLL_2511533004 head,
// buat sebuah node dengan sebuah nilai
NodeSLL_2511533004 newNode = new NodeSLL_2511533004(value);
// jika list kosong maka node jadi head
if (head == null) {
return newNode;
}

```

Jika list kosong, node langsung menjadi head dan variabel membantu membuat jalan dari head. Loop sampai ke node terakhir, node terakhir diarahkan ke node baru, dan head tetap (tidak berubah).

```

// simpan head ke variabel sementara
NodeSLL_2511533004 last = head;
// telusuri ke node akhir
while (last.next_3004 != null) {
last = last.next_3004;
}
// ubah pointer
last.next_3004 = newNode;
return head;
}

```

Fungsi sederhana untuk membuat node baru agar lebih ringkas dipanggil pada fungsi lain.

```

static NodeSLL_2511533004 GetNode(int data_3004) {
return new NodeSLL_2511533004(data_3004);
}

```


Fungsi untuk menambahkan node di posisi tertentu dengan menyimpan head awal, jika posisi kurang dari 1 berarti tidak valid, jika posisi 1 berarti sama seperti insert depan..

```
static NodeSLL_2511533004 insertPos(NodeSLL_2511533004 headNode, int p, NodeSLL_2511533004 head) {
    head = headNode;

    if (position < 1) {
        System.out.print("Invalid position");
    }

    if (position == 1) {
        NodeSLL_2511533004 new_node = new NodeSLL_2511533004(value);
        new_node.next_3004 = head;
        return new_node;
    }
}
```

Jika posisi bukan 1, loop mengurangi posisi, saat sudah sampai sebelum posisi target, membuat node baru, dan node baru diarahkan ke node setelahnya, node sebelumnya diarahkan ke node baru, dan keluar loop.

```
while (position-- != 0) {
    if (position == 1) {
        NodeSLL_2511533004 newNode = GetNode(value);
        newNode.next_3004 = headNode.next_3004;
        headNode.next_3004 = newNode;
        break;
    }
    headNode = headNode.next_3004;
}
```

Jika posisi melebihi panjang list berarti error, balikin ke head awal.

```
if (position != 1) {
    System.out.print("Posisi di luar jangkauan");
}
return head;
}
```

Fungsi untuk menampilkan isi linked list, mulai dari head, loop sampai sebelum node terakhir, print data dan panah, geser ke node berikutnya, jika sudah node terakhir, print tanpa panah..

```
public static void printList(NodeSLL_2511533004 head) {
    NodeSLL_2511533004 curr = head;
    while (curr.next_3004 != null) {
        System.out.print(curr.data_3004 + "-->");
        curr = curr.next_3004;
    }

    if (curr.next_3004 == null) {
        System.out.print(curr.data_3004);
    }
    System.out.println();
}
```

Membuat linked list awal. Node pertama: 2, sehingga list:
2 => 3 => 5 => 6.

```
NodeSLL_2511533004 head = new NodeSLL_2511533004(2);
head.next_3004 = new NodeSLL_2511533004(3);
head.next_3004.next_3004 = new NodeSLL_2511533004(5);
head.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(6);
```

Cetak list awal dan menambahkan di depan sehingga menjadi

1 => 2 => 3 => 5 => 6.

```
System.out.print("Senarai berantai awal:");
printList(head);
// tambahkan node baru di depan
System.out.print("tambah 1 simpul di depan: ");
int data = 1;
head = insertAtFront(head, data);
```

Menambahkan di belakang sehingga menjadi Menambahkan di depan sehingga menjadi

1 => 2 => 3 => 5 => 6 => 7.

```
System.out.print("tambah 1 simpul di belakang: ");
int data2 = 7;
head = insertAtEnd(head, data2);
// cetak update list
```

Menambahkan angka 4 pada posisi ke-4.

```
printList(head);
System.out.print("tambah 1 simpul ke data 4: ");
int data3 = 4;
int pos = 4;
head = insertPos(head, pos, data3);
// cetak update list
printList(head);
}
```

Output

```
Senarai berantai awal:2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7
```

2.3 PencarianSLL_2511533004

Kode Program

```
1 package pekan5_2511533004;
2
3 public class PencarianSLL_2511533004 {
4     static boolean searchKey(NodeSLL_2511533004 head, int key) {
5         NodeSLL_2511533004 curr = head;
6         while (curr != null) {
7             if (curr.data_3004 == key)
8                 return true;
9             curr = curr.next_3004;
10        }
11        return false;
12    }
13    public static void traversal(NodeSLL_2511533004 head) {
14        //mulai dari head
15        NodeSLL_2511533004 curr = head;
16        //telusuri sampai pointer null
17        while (curr != null) {
18            System.out.print(" " + curr.data_3004);
19            curr = curr.next_3004;
20        }
21        System.out.println();
22    }
23    public static void main(String[] args) {
24        NodeSLL_2511533004 head = new NodeSLL_2511533004(14);
25        head.next_3004 = new NodeSLL_2511533004(21);
26        head.next_3004.next_3004 = new NodeSLL_2511533004(13);
27        head.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(30);
28        head.next_3004.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(10);
29        System.out.print("Penelusuran SLL : ");
30        traversal(head);
31        // data yang akan dicari
32        int key = 30;
33        System.out.print("cari data " + key + " = ");
34        if (searchKey(head, key))
35            System.out.println("ketemu");
36        else
37            System.out.println("tidak ada");
38    }
}
```

Langkah Kerja

Fungsi dibuat untuk mencari apakah suatu nilai ada di linked list atau tidak. Alurnya mulai dari head (node pertama), selama node belum habis (belum null), terus dicek satu-satu. Jika data di node sama dengan yang dicari maka langsung ketemu (true). Jika belum ketemu maka pindah ke node berikutnya. Jika sudah sampai akhir tapi tidak ada, maka false (tidak ditemukan).

```
static boolean searchKey(NodeSLL_2511533004 head, int key) {
    NodeSLL_2511533004 curr = head;
    while (curr != null) {
        if (curr.data_3004 == key)
            return true;
        curr = curr.next_3004;
    }
    return false;
}
```

Fungsi ini dibuat untuk menampilkan semua isi linked list. Alurnya mulai dari head, selama node masih ada, tampilkan data. Geser ke node berikutnya lalu pindah aris setelah selesai.

```
public static void traversal(NodeSLL_2511533004 head) {
    //mulai dari head
    NodeSLL_2511533004 curr = head;
    //telusuri sampai pointer null
    while (curr != null) {
        System.out.print(" " + curr.data_3004);
        curr = curr.next_3004;
    }
    System.out.println();
}
```

Membuat linked list, node pertama: 14, sehingga linked list:
14 => 21 => 13 => 30 => 10.

```
NodeSLL_2511533004 head = new NodeSLL_2511533004(14);  
head.next_3004 = new NodeSLL_2511533004(21);  
head.next_3004.next_3004 = new NodeSLL_2511533004(13);  
head.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(30);  
head.next_3004.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(10);  
System.out.print("Penelusuran SLL : ");  
traversal(head);  
// data yang akan dicari  
int key = 30;  
System.out.print("cari data " + key + " = ");
```

Menampilkan isi list.

```
System.out.print("cari data " + key + " = ");  
if (searchKey(head, key))
```

Menentukan data yang ingin dicari dan menampilkan teks dilayar. Program mulai memanggil fungsi pencarian mulai dari node pertama (head) dan dicek satu persatu, jika ketemu, fungsi langsung mengembalikan true. Jika data tidak ditemukan, maka false.

```
int key = 30;  
System.out.print("cari data " + key + " = ");  
if (searchKey(head, key))  
    System.out.println("ketemu");  
else  
    System.out.println("tidak ada");
```

Output

```
Penelusuran SLL : 14 21 13 30 10  
cari data 30 = ketemu
```

2.4 HapusSLL_2511533004

Kode Program

```
1 package pekan3_2511533004;
2
3 public class HapusSLL_2511533004 {
4     // fungsi untuk menghapus head
5     public static NodeSLL_2511533004 deleteHead(NodeSLL_2511533004 head) {
6         // jika SLL kosong
7         if (head == null)
8             return null;
9         // pindahkan head ke node berikutnya
10        head = head.next_3004;
11        // Return head baru
12        return head;
13    }
14    // fungsi menghapus node terakhir SLL
15    public static NodeSLL_2511533004 removeLastNode(NodeSLL_2511533004 head) {
16        // jika list kosong, return null
17        if (head == null) {
18            return null;
19        }
20        // jika list satu node, hapus node dan return null
21        if (head.next_3004 == null) {
22            return null;
23        }
24        // temukan node terakhir ke dua
25        NodeSLL_2511533004 secondLast = head;
26        while (secondLast.next_3004.next_3004 != null) {
27            secondLast = secondLast.next_3004;
28        }
29        // hapus node terakhir
30        secondLast.next_3004 = null;
31        return head;
32    }
33    // fungsi menghapus node di posisi tertentu
34    public static NodeSLL_2511533004 deleteNode(NodeSLL_2511533004 head, int position) {
35        NodeSLL_2511533004 temp = head;
36        NodeSLL_2511533004 prev = null;
37
38        // jika linked list null
39        if (temp == null)
40            return head;
41
42        // kasus 1: head dihapus
43        if (position == 1) {
44            head = temp.next_3004;
45            return head;
46        }
47
48        // kasus 2: menghapus node di tengah
49        // telusuri ke node yang dihapus
50        for (int i = 1; temp != null && i < position; i++) {
51            prev = temp;
52            temp = temp.next_3004;
53        }
54
55        // jika ditemukan, hapus node
56        if (temp != null) {
57            prev.next_3004 = temp.next_3004;
58        } else {
59            System.out.println("Data tidak ada");
60        }
61        return head;
62    }
63    // fungsi mencetak SLL
64    public static void printList(NodeSLL_2511533004 head) {
65        NodeSLL_2511533004 curr = head;
66        while (curr.next_3004 != null) {
67            System.out.print(curr.data_3004 + "-->");
68            curr = curr.next_3004;
69        }
70
71        if (curr.next_3004 == null) {
72            System.out.print(curr.data_3004);
73        }
74        System.out.println();
75    }
76    // kelas main
77    public static void main(String[] args) {
78        // buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
79        NodeSLL_2511533004 head = new NodeSLL_2511533004(1);
80        head.next_3004 = new NodeSLL_2511533004(2);
81        head.next_3004.next_3004 = new NodeSLL_2511533004(3);
82        head.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(4);
83        head.next_3004.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(5);
84        head.next_3004.next_3004.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(6);
85        // cetak list awal
86        System.out.println("list awal: ");
87        printList(head);
88        // hapus head
89        head = deleteHead(head);
90        System.out.println("List setelah head dihapus: ");
91        printList(head);
92        // hapus node terakhir
93        head = removeLastNode(head);
94        System.out.println("List setelah simpul terakhir di hapus: ");
95        printList(head);
96        // Deleting node at position 2
97        int position = 2;
98        head = deleteNode(head, position);
99        // Print list after deletion
100        System.out.println("List setelah posisi 2 dihapus: ");
101        printList(head);
102    }
103 }
```

Langkah Kerja

Jika list kosong, maka langsung selesai (tidak ada yang dihapus). Jika ada isi, head digeser ke node berikutnya dan node pertama otomatis “kehapus” (karena ga direferensiin lagi), lalu balikin ke head baru

```
public static NodeSLL_2511533004 deleteHead(NodeSLL_2511533004 head) {  
    // jika SLL kosong  
    if (head == null)  
        return null;  
    // pindahkan head ke node berikutnya  
    head = head.next_3004;  
    // Return head baru  
    return head;  
}
```

Jika list kosong dan cuma 1 node, maka langsung kosong (node itu dihapus).

```
public static NodeSLL_2511533004 removeLastNode(NodeSLL_2511533004 head) {  
    // jika list kosong, return null  
    if (head == null) {  
        return null;  
    }  
    // jika list satu node, hapus node dan return null  
}
```

Mencari node sebelum terakhir, berhenti di node ke-2 terakhir. Pointer diputus dan node terakhir hilang.

```
// NodeSLL_2511533004 secondLast = head;  
while (secondLast.next_3004.next_3004 != null) {  
    secondLast = secondLast.next_3004;  
}  
// hapus node terakhir  
secondLast.next_3004 = null;  
return head;
```

Inisialisasi

```
public static NodeSLL_2511533004 deleteNode(NodeSLL_2511533004 head, int position)  
    NodeSLL_2511533004 temp = head;  
    NodeSLL_2511533004 prev = null;
```

List kosong dan hapus posisi 1 (head).

```
// jika linked list null  
if (temp == null)  
    return head;  
  
// kasus 1: head dihapus  
if (position == 1) {  
    head = temp.next_3004;  
    return head;  
}
```

Jika hapus ditengah, loop sampai node yang mau dihapus.

```
for (int i = 1; temp != null && i < position; i++) {  
    prev = temp;  
    temp = temp.next_3004;  
}
```

Node sebelumnya langsung lompat ke node setelahnya dan node ditengah kehapus dan jika posisi melebihi panjang list, maka "Data tidak ada". if (temp_3004 != null)

```
{
    if (temp != null) {
        prev.next_3004 = temp.next_3004;
    } else {
        System.out.println("Data tidak ada");
    }
    return head;
}
```

Menampilkan isi linked list.

```
while (curr.next_3004 != null) {
    System.out.print(curr.data_3004 + "-->");
    curr = curr.next_3004;
}

if (curr.next_3004 == null) {
    System.out.print(curr.data_3004);
}
```

Membuat linked list dan menampilkan list awal.

```
NodeSLL_2511533004 head = new NodeSLL_2511533004(1);
head.next_3004 = new NodeSLL_2511533004(2);
head.next_3004.next_3004 = new NodeSLL_2511533004(3);
head.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(4);
head.next_3004.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(5);
head.next_3004.next_3004.next_3004.next_3004.next_3004 = new NodeSLL_2511533004(6);
// cetak list awal
System.out.println("list awal: ");
```

10 Node 1 dihapus, head pindah ke 2.

```
// xxxxxx head
head = deleteHead(head);
System.out.println("List setelah head dihapus: ");
printList(head);
```

Menghapus node terakhir, node 6 dihapus.

```
head = removeLastNode(head);
System.out.println("List setelah simpul terakhir di hapus: ");
printList(head);
// Deleting node at position 2
```

Menghapus node di posisi ke-2.

```
int position = 2;
head = deleteNode(head, position);
// Print list after deletion
System.out.println("List setelah posisi 2 dihapus: ");
printList(head);
}
```

Output

```
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

2.5 Tugas

Class Pasien_2511533004

```
1 package pekan5_2511533004;
2
3 public class PasienSLL_2511533004 {
4     private String namaPasien_3004;
5     private String penyakit_3004;
6     private int nomorAntrian_3004;
7     PasienSLL_2511533004 next_3004;
8
9     public PasienSLL_2511533004(String namaPasien_3004, String penyakit_3004, int nomorAntrian_3004) {
10         this.namaPasien_3004 = namaPasien_3004;
11         this.penakit_3004 = penyakit_3004;
12         this.nomorAntrian_3004 = nomorAntrian_3004;
13         this.next_3004 = null;
14     }
15
16     public String getNamaPasien_3004() { return namaPasien_3004; }
17     public String getPenyakit_3004() { return penyakit_3004; }
18     public int getNomorAntrian_3004() { return nomorAntrian_3004; }
19     public PasienSLL_2511533004 getNext_3004() { return next_3004; }
20
21     public void setNext_3004(PasienSLL_2511533004 next_3004) { this.next_3004 = next_3004; }
22 }
```

Class RumahSakit_2511533004

```
1 package pekan5_2511533004;
2
3 import java.util.Scanner;
4
5 public class RumahSakitSLL_2511533004 {
6
7     private PasienSLL_2511533004 head_3004;
8     private int counter_3004;
9     // constructor
10    public RumahSakitSLL_2511533004() {
11        head_3004 = null;
12        counter_3004 = 0;
13    }
14    // Tambah Pasien
15    public void daftarkanPasien_3004(String nama_3004, String penyakit_3004) {
16
17        counter_3004++;
18
19        PasienSLL_2511533004 newPasien_3004 =
20            new PasienSLL_2511533004(
21                nama_3004,
22                penyakit_3004,
23                counter_3004
24            );
25
26        // jika antrian kosong
27        if (head_3004 == null) {
28            head_3004 = newPasien_3004;
29        } else {
30
31            // cari node terakhir
32            PasienSLL_2511533004 temp_3004 = head_3004;
33
34            while (temp_3004.getNext_3004() != null) {
35                temp_3004 = temp_3004.getNext_3004();
36            }
37
38            // tambahkan node baru
39            temp_3004.setNext_3004(newPasien_3004);
40        }
41
42        System.out.println("Pasien berhasil didaftarkan!");
43    }
44
45    System.out.println("Nomor Antrian : " + counter_3004);
46
47    // Panggil Pasien
48    public void panggilPasien_3004() {
49
50        if (head_3004 == null) {
51            System.out.println("Antrian kosong!");
52            return;
53        }
54        System.out.println("\n=== Memanggil Pasien ===");
55        System.out.println("Nama : " + head_3004.getNamaPasien_3004());
56        System.out.println("Keluhan : " + head_3004.getPenyakit_3004());
57
58        // hapus node pertama
59        head_3004 = head_3004.getNext_3004();
60    }
61
62    // Tampilkan Antrian
63    public void tampilkanAntrian_3004() {
64
65        if (head_3004 == null) {
66            System.out.println("Antrian kosong!");
67            return;
68        }
69        PasienSLL_2511533004 temp_3004 = head_3004;
70
71        System.out.println("\n=== Daftar Antrian Pasien ===");
72
73        while (temp_3004 != null) {
74
75            System.out.println(
76                "Nomor : " + temp_3004.getNomorAntrian_3004()
77                + " | Nama : " + temp_3004.getNamaPasien_3004()
78                + " | Keluhan : " + temp_3004.getPenyakit_3004()
79            );
80
81            temp_3004 = temp_3004.getNext_3004();
82        }
83    }
84
85    // cari pasien
86 }
```



```

86 // Cari Pasien
87 public void cariPasien_3004(String nama_3004) {
88
89     PasienSLL_2511533004 temp_3004 = head_3004;
90
91     while (temp_3004 != null) {
92
93         if (temp_3004.getNamaPasien_3004().equalsIgnoreCase(nama_3004)) {
94
95             System.out.println("\nPasien ditemukan!");
96             System.out.println("Nomor Antrian : " + temp_3004.getNomorAntrian_3004());
97             System.out.println("Nama : " + temp_3004.getNamaPasien_3004());
98             System.out.println("Keluhan : " + temp_3004.getPenyakit_3004());
99
100             return;
101         }
102         temp_3004 = temp_3004.getNext_3004();
103     }
104
105     System.out.println("Pasien tidak ditemukan!");
106 }
107
108 // Cek status antrian
109 public void cekStatusAntrian_3004() {
110
111     if (head_3004 == null) {
112
113         System.out.println("Antrian kosong!");
114         return;
115     }
116
117     int jumlah_3004 = 0;
118
119     PasienSLL_2511533004 temp_3004 = head_3004;
120
121     while (temp_3004 != null) {
122
123         jumlah_3004++;
124         temp_3004 = temp_3004.getNext_3004();
125     }
126
127     System.out.println("\nJumlah pasien : " + jumlah_3004);
128
129     System.out.println(
130         "Pasien terdepan : "
131         + head_3004.getNamaPasien_3004()
132         + " | Keluhan : "
133         + head_3004.getPenyakit_3004()
134     );
135 }
136
137 // Main Program
138 public static void main(String[] args) {
139
140     Scanner sc_3004 = new Scanner(System.in);
141
142     RumahSakitSLL_2511533004 rs_3004 =
143         new RumahSakitSLL_2511533004();
144
145     int pilihan_3004;
146
147     do {
148
149         System.out.println("\n== Antrian Rumah Sakit SLL NIM: 2511533004 ==");
150         System.out.println("1. Daftarkan Pasien");
151         System.out.println("2. Panggil Pasien");
152         System.out.println("3. Tampilkan Antrian");
153         System.out.println("4. Cari Pasien");
154         System.out.println("5. Cek Status Antrian");
155         System.out.println("6. Keluar");
156
157         System.out.print("Pilihan : ");
158
159         pilihan_3004 = sc_3004.nextInt();
160         sc_3004.nextLine();
161
162         switch (pilihan_3004) {
163
164             case 1:
165
166                 System.out.print("Masukkan Nama Pasien : ");
167                 String nama_3004 = sc_3004.nextLine();
168
169                 System.out.print("Masukkan Keluhan : ");
170                 String penyakit_3004 = sc_3004.nextLine();
171
172                 rs_3004.daftarkanPasien_3004(
173                     nama_3004,
174                     penyakit_3004
175                 );
176                 break;
177
178             case 2:
179
180                 rs_3004.panggilPasien_3004();
181                 break;
182
183             case 3:
184
185                 rs_3004.tampilkanAntrian_3004();
186                 break;
187
188             case 4:
189
190                 System.out.print("Masukkan Nama Pasien yang dicari : ");
191
192                 String cari_3004 = sc_3004.nextLine();
193
194                 rs_3004.cariPasien_3004(cari_3004);
195                 break;
196
197             case 5:
198
199                 rs_3004.cekStatusAntrian_3004();
200                 break;
201
202             case 6:
203
204                 System.out.println("Program selesai.");
205                 break;
206
207             default:
208
209                 System.out.println("Pilihan tidak valid!");
210
211         }
212     } while (true);

```

```

213     }
214     } while (pilihan_3004 != 6);
215     } while (pilihan_3004 != 6);
216     sc_3004.close();
217 }
218 }
219 }

```

Output

1. Menambahkan pasien.

```

=== Antrian Rumah Sakit SLL NIM: 2511533004 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : silva
Masukkan Keluhan : pusing
Pasien berhasil didaftarkan!
Nomor Antrian : 1

=== Antrian Rumah Sakit SLL NIM: 2511533004 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : jefan
Masukkan Keluhan : mual
Pasien berhasil didaftarkan!
Nomor Antrian : 2

=== Antrian Rumah Sakit SLL NIM: 2511533004 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 1
Masukkan Nama Pasien : nay
Masukkan Keluhan : demam
Pasien berhasil didaftarkan!
Nomor Antrian : 3

```

Output diatas menunjukkan bagaimana program dapat menaftrakan nama pasien dengan memilih menu 1. User diminta untuk mengisi nama pasien serta keluhan yang dialami oleh pasien. Setelah data di input, maka pasien berhasil didaftarkan dan mendapat nomor antrian sesuai dengan urutan daftar pasien.

2. Menampilkan antrian.

```

=== Daftar Antrian Pasien ===
Nomor : 1 | Nama : silva | Keluhan : pusing
Nomor : 2 | Nama : jefan | Keluhan : mual
Nomor : 3 | Nama : nay | Keluhan : demam

```

Output diatas menunjukkan bagaimana program dapat menampilkan antrian yang tersedia. Antrian tersebut berisi pasien pasien yang telah didaftarkan sebelumnya.

3. Hbedygdeuyggde

```
=== Antrian Rumah Sakit SLL NIM: 2511533004 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 4
Masukkan Nama Pasien yang dicari : silva

Pasien ditemukan!
Nomor Antrian : 1
Nama : silva
Keluhan : pusing

=== Antrian Rumah Sakit SLL NIM: 2511533004 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status Antrian
6. Keluar
Pilihan : 4
Masukkan Nama Pasien yang dicari : jefan

Pasien ditemukan!
Nomor Antrian : 2
Nama : jefan
Keluhan : mual
```

Output diatas menunjukkan bagaimana program dapat mencari pasien berdasarkan nama. User dapat mencari pasien dengan menginput nama pasien, jika pasien ada dalam antrian, maka pasien ditemukan dan program mencetak nama pasien serta keluhan yang dialami oleh pasien. Namun, jika nama pasien tidak ditemukan, maka pasien telah dipanggil atau pasien tidak terdaftar dalam antrian.

4. Panggil pasien

```
=== Memanggil Pasien ===
Nama : silva
Keluhan : pusing
```

Output diatas menunjukkan program dapat memanggil pasien mulai dari antrian yang paling atas. Sistem ini menggunakan First In, First Out (FIFO). Ketika pasien dipanggil, program menampilkan nama pasien, keluhan, serta nomor antrian pasien.

5. Cek status antrian.

```
Jumlah pasien : 2
Pasien terdepan : jefan | Keluhan : mual
```

Output diatas menunjukkan program dapat mengecek status antrian. Ketika user memilih menu no 5, maka program menampilkan jumlah pasien yang masih tersedia dalam antrian. Program juga menampilkan pasien terdepan, yang dimana pasien berada pada antrian paling awal. Sehingga ketika user memanggil pasien, maka pasien tersebut yang dipanggil di awal.

BAB III

PENUTUPAN

Single Linked List (SLL) merupakan struktur data dinamis yang tersusun dari node-node yang saling terhubung melalui pointer. Setiap node memiliki data dan referensi ke node berikutnya, sehingga memungkinkan pengelolaan data secara fleksibel. Melalui penggunaan class NodeSLL, dapat dibangun struktur dasar linked list. Operasi penambahan node yang diimplementasikan dalam TambahSLL menunjukkan bahwa data dapat disisipkan di berbagai posisi tanpa perlu menggeser elemen lain. Selanjutnya, proses pencarian pada PencarianSLL dilakukan dengan menelusuri node secara berurutan hingga data ditemukan. Sementara itu, pada HapusSLL, penghapusan node dilakukan dengan mengubah hubungan antar node sehingga struktur list tetap terjaga. Secara keseluruhan, pemahaman terhadap operasi penambahan, pencarian, dan penghapusan pada Single Linked List membantu dalam mengelola data secara lebih efisien dan menjadi dasar untuk mempelajari struktur data yang lebih kompleks.